

Aprender a lançar — o que foi feito, e porquê

Este documento explica, passo a passo, o que se fez para pôr o AYContabilidade a correr na Internet. Não é uma lista de comandos para copiar às cegas: cada um vem com **o que faz**, **porque é preciso** e **como se confirma que resultou**.

Escrito a partir do lançamento real de 16 de Agosto de 2026.

1. As três peças, e porque são três

Uma aplicação como esta não é um ficheiro que se põe num sítio. São três coisas diferentes, que podem viver em sítios diferentes:

Peça	O que é	Onde ficou
Frontend	O que se vê no browser. Next.js.	Render
Backend	Quem sabe as regras e fala com a base. FastAPI.	Render
Base de dados	Onde ficam os dados. PostgreSQL.	Neon

Porque não tudo no mesmo sítio: porque cada um tem exigências diferentes. A base de dados tem de guardar coisas para sempre e não pode ser reiniciada à vontade; o frontend e o backend são descartáveis — apaga-se e volta a construir-se do código. Misturá-los é o que faz com que um dia se perca a base ao reinstalar a aplicação.

2. O código tem de estar num sítio que o servidor alcance

O Render não lê o disco da sua máquina. Lê o **GitHub**. Por isso o primeiro passo é o código estar lá.

```
git remote -v
```

Mostra para onde o `git push` envia. Se aparecer o repositório errado:

```
git remote set-url origin https://github.com/UTILIZADOR/REPOSITARIO.git
```

E depois:

```
git push -u origin main
```

Se falhar com HTTP 408 — foi o que aconteceu aqui — o envio é grande de mais para o tamanho de bloco por omissão do git. Resolve-se uma vez:

```
git config http.postBuffer 524288000
git config http.version HTTP/1.1
```

E repete-se o `push`.

Como se confirma:

```
git ls-remote --heads origin
```

Se devolver uma linha com `refs/heads/main`, está lá. Se não devolver nada, o repositório existe mas está vazio — o `push` não passou.

> **O repositório deve ser privado** se o código não é seu para publicar. > GitHub → Settings → General → Danger Zone → Change visibility.

3. A base de dados primeiro, sempre

Ordem: base → migrações → primeira conta → serviços. Ao contrário, o backend sobe e morre à procura de uma base que não existe.

No Neon: **New Project**, e três escolhas que importam:

- **Postgres 18** — é a versão para que o sistema foi feito.
- **Região igual à do backend.** Aqui, Frankfurt nos dois. Com a base em Oregon

e o backend em Frankfurt, cada consulta atravessa o Atlântico duas vezes: ~150 ms em vez de ~2 ms. Um ecrã com dez consultas fica um segundo e meio mais lento, e há ecrãs com mais de dez.

- **Neon Auth desligado** — a aplicação já tem contas, sessões e segundo

factor. Ligar aquilo era pôr lá um segundo sistema de contas sem uso.

No fim, o Neon dá a **connection string**:

```
postgresql://utilizador:PALAVRA_PASSE@ep-xxxx.eu-central-1.aws.neon.tech/neondb?sslmode=require
```

4. Onde se guarda uma linha destas (e onde NÃO se guarda)

Aquela linha tem a palavra-passe da base lá dentro. Quem a tiver, tem os dados.

Onde não vai: não vai para o código, não vai para o git, não se cola numa conversa, e é melhor não ir para o histórico da shell.

O que se fez aqui: um ficheiro próprio, que o git ignora.

```
# Producao/backend/.env.neon
DATABASE_URL=postgresql://...neon.tech/neondb?sslmode=require
```

Confirma-se que o git o ignora mesmo:

```
git check-ignore -v Producao/backend/.env.neon
```

Se responder com a linha do `.gitignore` que o apanha, está seguro. Se não responder nada, **o ficheiro ia para o GitHub** — pare e corrija o `.gitignore` antes de continuar.

Para usar o ficheiro sem o escrever no ecrã:

```
set -a && . ./env.neon && set +a
```

`set -a` faz com que tudo o que se defina a seguir seja exportado para os programas que se chamem; o `.` (ponto) lê o ficheiro; `set +a` volta ao normal. A partir daqui, os comandos nesta janela do terminal conhecem a `DATABASE_URL` sem ela aparecer em lado nenhum.

5. Criar as tabelas — migrações

A base nasce vazia. As tabelas são criadas pelas **migrações**: ficheiros numerados que descrevem cada alteração à estrutura desde o princípio do projecto.

```
cd Producao/backend
set -a && . ./env.neon && set +a
./venv/Scripts/python.exe -m alembic upgrade head
```

`head` significa «até à última». O Alembic vê em que ponto a base está, compara com os ficheiros que existem, e aplica só o que falta. Correr isto duas vezes não faz mal nenhum: da segunda não tem nada a fazer.

Como se confirma — 41 tabelas e a migração no topo:

```
./venv/Scripts/python.exe -c "
from sqlalchemy import create_engine, text
from src.db.base import url_do_motor
import os
e = create_engine(url_do_motor(os.environ['DATABASE_URL']))
with e.connect() as c:
    n = c.execute(text('select count(*) from information_schema.tables where
table_schema='public')).scalar()
    v = c.execute(text('select version_num from alembic_version')).scalar()
    print('tabelas:', n, '| migração:', v)
"
```

6. A primeira conta

Não há nenhuma. A aplicação não traz contas de fábrica de propósito: uma conta com palavra-passe conhecida numa instalação real é uma porta aberta.

```
cd Producao/backend
set -a && . ./env.neon && set +a
./venv/Scripts/python.exe scripts/criar_superadmin.py
```

Pede nome, e-mail e palavra-passe **na consola**. Não aceita a palavra-passe por argumento, e é de propósito: o que se passa num argumento fica no histórico da shell e na lista de processos, onde qualquer outra sessão da máquina o vê.

Este comando é o único que tem mesmo de correr você. Ninguém deve saber essa palavra-passe além de si.

> **Nunca corra o `criar_demo.py` numa base a sério.** Ele recusa-se em > produção, e faz bem: cria contas com palavras-passe conhecidas.

7. Os serviços no Render

O ficheiro `render.yaml`, na raiz do repositório, descreve os dois serviços. No Render é **New** → **Blueprint** → escolher o repositório. Ele lê o ficheiro.

O que o ficheiro diz, por serviço:

Campo	Backend	Frontend
<code>rootDir</code>	<code>Producao/backend</code>	<code>Producao/frontend</code>
<code>buildCommand</code>	<code>pip install -r requirements.txt</code>	<code>npm ci && npm run build && cp -r ...</code>
<code>startCommand</code>	<code>uvicorn main:app --host 0.0.0.0 --port \$PORT</code>	<code>node .next/standalone/server.js</code>
<code>healthCheckPath</code>	<code>/api/health</code>	<code>/entrar</code>

Três coisas que se aprendem à custa de as errar:

`$PORT` **não se inventa**. O Render escolhe a porta e diz-a nesta variável. Fixar 8001 faz o serviço subir e nunca receber um pedido.

A sonda de saúde tem de devolver 200. Estava apontada a `/docs` — que *fecha* em produção e devolve 404. O Render leria «serviço doente» e reiniciava-o em ciclo. `/api/health` é a única rota sem autenticação, e existe para isto.

O output: `standalone` do Next não copia tudo. Faltam o `.next/static` e a `public/`, e sem as duas cópias no build o site sobe sem CSS nenhum e os ficheiros de `public/` dão 404. Daí as duas linhas de `cp` no `buildCommand`.

8. O nó que apanha toda a gente: os dois endereços

O backend só aceita pedidos do frontend se souber o endereço dele (`CORS_ORIGINS`). O frontend só sabe falar com o backend se souber o endereço dele (`NEXT_PUBLIC_API_URL`). E nenhum dos dois existe antes de estarem criados.

Faz-se assim:

1. Criar os dois com os endereços **previstos**

(`https://aycontabilidade-api.onrender.com` e `...-web.onrender.com`).

1. Depois de criados, **confirmar os endereços reais**. Se o nome já estivesse ocupado, o Render acrescenta um sufixo e o que se pôs está errado.

1. Corrigir as duas variáveis e **reconstruir o frontend**.

A reconstrução não é opcional: tudo o que começa por `NEXT_PUBLIC_` é lido **quando se constrói**, não quando se arranca. Reiniciar o serviço mantém o endereço antigo lá dentro.

Sintomas, para reconhecer sem adivinhar:

- Ecrã carrega mas nada tem dados, e a consola do browser diz `CORS` →

`CORS_ORIGINS` errado no backend.

- Ecrã carrega e os pedidos vão para `localhost:8001` → `NEXT_PUBLIC_API_URL`

errado, ou o frontend não foi reconstruído depois de o corrigir.

- Backend nem arranca, e o registo diz que não arranca em `AMBIENTE=producao`

→ é a rede de segurança da aplicação. A mensagem diz qual das condições falhou; não se contorna, corrige-se.

9. O que o plano gratuito custa

- **Adormece** com 15 minutos sem pedidos. O primeiro pedido a seguir demora

cerca de **um minuto**. Não é avaria.

- **750 horas de serviço por mês** por conta, somando os serviços.
- O Neon adormece em 5 minutos e acorda em menos de um segundo.
- **Não há cópias de segurança**. Numa avaliação com dados de uma empresa a

sério, uma cópia por semana é o mínimo:

```
pg_dump "postgresql://...neon.tech/neondb?sslmode=require" -Fc -f copia-2026-08-16.dump
```

10. A ordem, em nove linhas

- | | |
|---|---|
| 1. Código no GitHub (privado) | <code>git push -u origin main</code> |
| 2. Base no Neon | Postgres 18, mesma região do backend |
| 3. Ligação num ficheiro ignorado | <code>.env.neon</code> |
| 4. Tabelas | <code>alembic upgrade head</code> |
| 5. Primeira conta | <code>scripts/criar_superadmin.py</code> |
| 6. Serviços | Render → Blueprint |
| 7. Endereços reais | corrigir <code>CORS_ORIGINS</code> e <code>NEXT_PUBLIC_API_URL</code> |
| 8. Reconstruir o frontend | obrigatório após mudar <code>NEXT_PUBLIC_*</code> |
| 9. Entrar, activar o 2FA, criar a licença e a empresa | |

Guarde à parte, fora do Render, a `TOTP_CHAVE_CIFRA`. Se se perder, todas as contas com segundo factor têm de o configurar outra vez — e a administração da plataforma exige segundo factor.